# Minutes of the Meeting

**Source File:** TTS 1 5mins.mp4
**Date:** May 03, 2026
**Meeting Duration:** 5m 6s

## Executive Overview

The meeting focused on the final stretch of the students' academic career, emphasizing the transition from building temporary digital solutions to designing long-lasting systems. The instructor discussed the importance of systems architecture, comparing it to building a cathedral versus a shack. The main assignment, titled "Assignment No. 1," requires students to design a system that handles 10,000 concurrent data streams while maintaining a zero-latency source of truth.

To guide the students through this assignment, the instructor assigned three specific subtasks: Produce a logic map before writing any code. Handle edge cases, including power outages and malicious input. Simulate a load of 10,000 users and find the breaking point of the system. The instructor also emphasized the importance of writing code that is easy to understand, maintain, and debug, and warned against using clever code that may be difficult to maintain.

## Topics Discussed

**1.** Transitioning from coding to systems architecture.

**2.** Designing a high-concurrency data handling system.

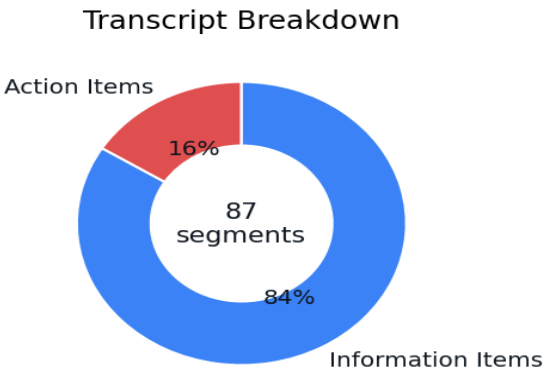
**3.** Writing robust and maintainable code under high pressure.

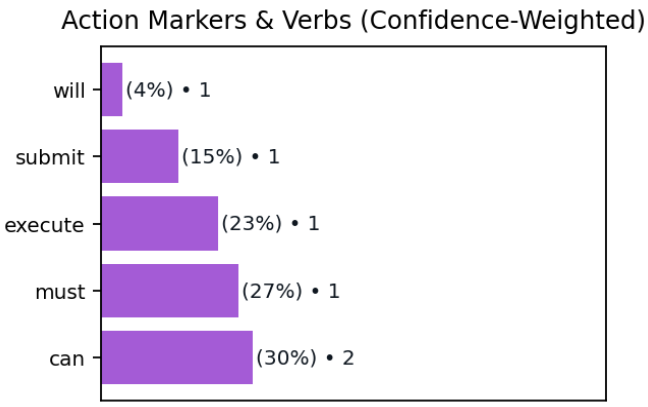
**4.** Transition from student to engineer through project work.

## Analytics Overview

Total sentences detected: 87. Suggested action items: 14. Information sentences: 73.

**Common action markers:** can (2, 30%), must (1, 27%), execute (1, 23%), submit (1, 15%), will (1, 4%)




## Action Items

**[ ]** Before you write a single line of code

Flagged because it describes a task or follow-up that implies completion.

**[ ]** I repeat

Flagged because it matches the classifier's action-item pattern without adding assumptions.

**[ ]** before you open your IDE

Flagged because it describes a task or follow-up that implies completion.

**[ ]** you must produce a logic map.

Flagged because the utterance contains a request or directive.

**[ ]** I want to see your edge cases handling. What happens when the power goes out mid-transaction? What happens when the input is malicious?

Flagged because it matches the classifier's action-item pattern without adding assumptions.

**[ ]** Assignment number one requires you to simulate a load. I don't want to see your program working for one user. I want to see it screaming under the pressure of 10

Flagged because it matches the classifier's action-item pattern without adding assumptions.

**[ ]** Code is for humans to read

Flagged because it matches the classifier's action-item pattern without adding assumptions.

**[ ]** only incidentally for machines to execute. If I can't understand your logic by glancing

Flagged because it describes a task or follow-up that implies completion.

**[ ]** your naming conventions

Flagged because it matches the classifier's action-item pattern without adding assumptions.

**[ ]** your structure

Flagged because it matches the classifier's action-item pattern without adding assumptions.

**[ ]** I will reject the submission.

Flagged because it matches the classifier's action-item pattern without adding assumptions.

**[ ]** I am not looking for completion. In the real world

Flagged because it matches the classifier's action-item pattern without adding assumptions.

**[ ]** a bridge that is 99% finished is a bridge that no one can cross.

Flagged because it matches the classifier's action-item pattern without adding assumptions.

**[ ]** You have exactly 336 hours from this moment to submit your repository link.

Flagged because it describes a task or follow-up that implies completion.

## Full Transcript

Good morning, everyone, please find your seats quickly. Close your laptops for just a moment. I want your eyes up here, not on a glowing rectangle. We are entering the final stretch of your academic career. By now, you know how to syntax check. You know how to library crawl. You know how to make a program run. But today, we stop talking about making things run and start talking about making things last. Most of what you have built in the last three years is digital sand. It's fragile. It's temporary. If a single variable changes in the environment, your code shatters. In the professional world, you aren't paid to write code. You are paid to manage complexity. You are paid to anticipate the stupidity of the user and the entropy of the machine. We call this systems architecture. It is the difference between a shack made of sticks and a cathedral made of stone One is easy to build but falls in a storm The other requires a blueprint a foundation and a deep understanding of physics Today we begin building your cathedral I have uploaded a file to the university server. It is titled,

simply, Assignment No. 1. If you open it, you will see it is surprisingly short. There are no step-by-step instructions. There are no hints sections. There isn't even a specified programming language. Some of you will find this frustrating. You will want me to tell you exactly which functions to call and which libraries to import. I won't do that. Your task is to complete assignment number one, the integrity protocol. The prompt asks you to design a system that handles 10,000 concurrent data streams while maintaining a zero latency source of truth. That is all. How you do it is up to you. Whether you use a monolithic structure or a microservices mesh is your decision. Whether you use SQL, NoSQL or flat files is your decision, but remember, every choice you make is a trade-off. If you choose speed, you might lose consistency. choose security you might lose performance. You cannot have it all. You must choose what kind of architect you are. To guide you through this vagueness, I am assigning three specific subtasks that I will be looking for during the code review. These are the action items that sit behind the main assignment. Before you write a single line of code, I repeat, before you open your IDE, you must produce a logic map. I want to see the flow of data. Where does it enter? Where is it validated. Where is it stored? If you start typing before you start thinking, you will trap yourself in a corner by midnight on day 3. I want to see your edge cases handling. What happens when the power goes out mid-transaction? What happens when the input is malicious? Assignment number one requires you to simulate a load. I don't want to see your program working for one user. I want to see it screaming under the pressure of 10,000. I want you to find your breaking point. Every system has one. A good engineer knows exactly where their bridge collapses. A bad engineer is surprised when it happens. Code is for humans to read and only incidentally for machines to execute. If I can't understand your logic by glancing at your naming conventions and your structure, I will reject the submission. Avoid clever code. Clever code is a nightmare to maintain. I want boring, predictable, stable code. I want code that looks like it was written by a professional, not a hobbyist. Let's talk about the grading. I am not looking for completion. In the real world, a bridge that is 99% finished is a bridge that no one can cross. If your integrity protocol fails once during my testing, just once, the assignment is a failure. There is no partial credit for data corruption. There is no A for effort when the database desyncs. This sounds harsh, I know, but in six months some of you will be writing code for medical devices or banking systems or autonomous vehicles. In those fields, almost right is a catastrophe. I am setting the bar high now so that you don't trip over it later. Assignment number one is your proof of concept. It proves to me, and more importantly, to yourself, that you are ready to move from being a student who follows recipes to an engineer who creates them. You have the remainder of this period to form your initial groups and begin the logic map phase. I will be walking around. Do not ask me if your idea is correct. Ask me if your idea is defensible. You must be able to defend every semicolon and every architectural pivot you make. The portal for assignment number 1 is open. The clock is ticking. You have exactly 336 hours from this moment to submit your repository link. Look to your left and your right. These are your colleagues. In this industry, your reputation starts here. If you are the person who doesn't contribute or the person who writes spaghetti code that others have to clean up, that reputation will follow you into the job market. Be the architect that others want to build with. That is all. Get to work. I'll see you in the lab.